

Initial Abstract Algorithmic Chemistry Experiments in PeTTa

A PeTTa-First, MORK-Ready Research and Implementation Plan

Prepared for human researchers and coding agents

June 2026

Abstract

This document specifies an initial research and software-development program for abstract algorithmic chemistry in PeTTa and an Atomspace/MeTTa context. The aim is to obtain measurable emergence of complex autocatalytic sets and associated evolutionary or ecological dynamics before introducing semantic graphs, MORK execution, PLN, MOSES, or music-composition-specific molecules. The first chemistry is deliberately abstract: molecules are small symbolic terms, token sequences, and later rule molecules; reactions are typed, bounded, catalytic rewrite rules over multisets. The design keeps clear seams for later acceleration or migration to MORK, especially for matching, sampling, canonicalization, bulk analytics, and structural rule evolution.

The document has two audiences. The main body is for human readers: it explains motivation, conceptual choices, scientific questions, implications, and a staged research trajectory from abstract chemistry to semantic and musical chemistry. The appendix is for coding agents: it gives step-by-step experiments, atom schemas, phase loops, analytics algorithms, repository layout, tests, acceptance criteria, and review checkpoints suitable for an OpenClaw-style implementation workflow using a coding model for routine development and intermittent strategic review by a stronger reasoning model.

Contents

1	Purpose and scope	5
2	Why start with abstract chemistry rather than semantic graphs	5
3	Relation to the ActPC-MORK and semantic-chemistry designs	6
4	Core scientific questions	6
4.1	Autocatalytic emergence	6
4.2	Evolutionary and ecological dynamics	6
4.3	Endogenous rule evolution	7
4.4	Prediction and active weighting	7
4.5	Causal relevance of ACSs	7
5	Recommended starting chemistry	7
5.1	Initial reaction families	7
5.2	Catalysis and inhibition	8

6	PeTTa-first architecture	8
6.1	Layer A: Atomspace state	8
6.2	Layer B: Phase engine	8
6.3	Layer C: Analytics	9
6.4	Layer D: Replaceable kernel boundary	9
7	Core atom conventions	9
7.1	Chambers and ticks	10
7.2	Molecules and abundances	10
7.3	Rules	10
7.4	Candidates and events	11
8	Analytics for autocatalytic sets	11
8.1	Descriptive chemistry metrics	11
8.2	Empirical catalysis mining	12
8.3	ACS and RAF-like closure	12
8.4	Causal relevance by replay and ablation	13
9	Experimental trajectory	13
9.1	Experiment 0: deterministic smoke test	13
9.2	Experiment 1: hand-planted ACS recovery	13
9.3	Experiment 2: random catalytic polymer chemistry	13
9.4	Experiment 3: rule molecules and endogenous rule evolution	14
9.5	Experiment 4: ecological chambers, migration, and parasites	14
9.6	Experiment 5: active-predictive weighting	14
9.7	Experiment 6: trace-guided structural mutation	14
9.8	Experiment 7: causal ACS validation archive	14
10	Expected outcomes and implications	14
11	Later path to semantic chemistry	15
12	Later path to music chemistry	15
13	Recommended implementation governance	16
A	Appendix: coding-agent implementation plan	17
A.1	Non-goals for version 0.1	17
B	Repository layout	17
C	Atom schema checklist	18
C.1	Chamber atoms	18
C.2	Molecule atoms	19
C.3	Rule atoms	19
C.4	Candidate and event atoms	19
C.5	Analytics atoms	20
D	Top-level tick pseudocode	20

E	Candidate matching guidance	21
F	Sampling guidance	21
G	ACS scanner pseudocode	22
H	Ablation pseudocode	23
I	Experiment 0: deterministic smoke test	24
I.1	Goal	24
I.2	Initial molecules	24
I.3	Hand-coded reactions	24
I.4	Acceptance criteria	24
J	Experiment 1: planted ACS recovery	24
J.1	Goal	24
J.2	Food set	24
J.3	Planted ACS	24
J.4	Run matrix	24
J.5	Metrics	25
K	Experiment 2: random catalytic polymer chemistry	25
K.1	Goal	25
K.2	Molecule space	25
K.3	Reaction families	25
K.4	Catalyst rules	25
K.5	Controls	25
K.6	Success criteria	25
L	Experiment 3: rule molecules	26
L.1	Goal	26
L.2	Rule molecule atoms	26
L.3	Rule mutation operators	26
L.4	Trial promotion	26
L.5	Metrics	26
M	Experiment 4: ecological chambers and parasites	27
M.1	Goal	27
M.2	Chamber setup	27
M.3	Parasite definition	27
M.4	Metrics	27
N	Experiment 5: active-predictive wrapper	27
N.1	Goal	27
N.2	Prediction target	27
N.3	Reward	28
N.4	Simple logit update	28
N.5	Acceptance criteria	28

O	Experiment 6: trace-guided structural mutation	28
O.1	Goal	28
O.2	Evidence sources	28
O.3	Mutation proposals	28
O.4	Comparison	28
P	Experiment 7: causal ACS archive	28
P.1	Goal	28
P.2	Archive record	29
P.3	Scientific report template	29
Q	Metrics file formats	29
R	Testing checklist	30
R.1	Unit tests	30
R.2	Integration tests	30
S	OpenClaw and model-review workflow	30
T	MORK migration seams	31
U	Failure modes and mitigations	31
V	Minimal configuration examples	32
V.1	Experiment 0	32
V.2	Experiment 1	32
V.3	Experiment 2	33
W	Stop conditions for coding agents	33
X	Bridge to semantic and music versions	33
X.1	Semantic bridge prototype	33
X.2	Music bridge prototype	33
Y	Bibliographic pointers	34

1 Purpose and scope

The immediate goal is not to build semantic graph chemistry, music chemistry, or a full ActPC-MORK stack. The immediate goal is to make a small but genuinely interesting algorithmic chemistry work in PeTTa alone: a chemical soup whose state is represented in an Atomspace, whose reaction rules are expressed in a MeTTa-like idiom, and whose run logs can be mined for autocatalytic structure, parasitism, succession, ecological niches, and causal relevance.

The work should begin with an abstract symbolic chemistry for four reasons. First, the abstractions make it possible to run many experiments quickly and to understand failures. Second, autocatalytic structure can be detected and tested without confounding it with natural-language parsing, semantic normalization, PLN truth-value propagation, or musical aesthetic scoring. Third, the same core software objects—molecules, reaction events, rule molecules, chambers, catalysts, analytics, ablations, and traces—will still be needed in semantic and musical versions. Fourth, a simple PeTTa-only implementation reduces integration risk while preserving a clean migration path to MORK.

The proposed first milestone is:

Demonstrate spontaneous, measurable, replayable, and intervention-relevant autocatalytic organization in a bounded abstract PeTTa chemistry.

This phrase is intentionally restrictive. “Spontaneous” means the most interesting autocatalytic sets should not be hand-coded, although the detector should first be validated on hand-planted examples. “Measurable” means the system must report diversity, recurrence, closure, persistence, productivity, and ablation impact. “Replayable” means all stochastic choices are seedable and all reaction events are logged. “Intervention-relevant” means candidate sets must be ablated, boosted, or shuffled to see whether they matter.

2 Why start with abstract chemistry rather than semantic graphs

Semantic graph chemistry is the long-term target, but it introduces several additional degrees of freedom at once: graph normalization, variable-bearing matching, contextual implication weights, task reward, truth-value dynamics, and natural-language realization. These are valuable later, but they obscure the first question: can the system cultivate useful recurrent reaction networks at all?

The abstract chemistry proposed here keeps the shape of the later system while removing domain complications. In the first version:

- Molecules are symbolic sequences or small trees rather than semantic graphs.
- Reaction rules are typed catalytic rewrites rather than PLN-weighted semantic transformations.
- Chambers are abstract ecological environments rather than discourse, biomedical, or musical contexts.
- Reward is persistence, productivity, complexity, prediction improvement, or controlled task score rather than question answering or aesthetic evaluation.
- Autocatalytic sets are detected from event logs and molecule-rule dependency graphs rather than from semantic inference loops.

This does not abandon semantic chemistry. It creates a substrate that can later host semantic graph fragments as molecules and contextual graph rewrites as reactions. The software should therefore avoid assumptions that molecules are strings forever. A molecule should be an opaque object with typed views, one of which may initially be a token sequence and later may be a graph, phrase, chord progression, or rule body.

3 Relation to the ActPC-MORK and semantic-chemistry designs

The ActPC-MORK design argues for a two-level architecture: a fast local inner loop over compact indexed atoms, and a slower structural loop that proposes new rules, abstractions, and meta-guards from traces and credit signals. This document keeps that separation but collapses it into PeTTa for the first implementation. MORK, MM2, MOSES, PLN, AIRIS, CENF, and optimal transport are not part of the first milestone.

The first implementation should nevertheless preserve their future interfaces. In particular:

- The inner loop should be phase-based and event-log-driven.
- Hot-path facts should be relation-first and context-scoped.
- Source rules and compiled matching facts should be separable, even if the first compiler is trivial.
- Stochastic sampling, scoring, ACS scanning, canonical hashing, and replay should be written behind replaceable request/response interfaces.
- New rules should enter as trial rules and be promoted or retired by evidence.
- Analytics should produce atoms, not merely external plots.

The semantic-graph chemistry design adds another important lesson: a loop is not interesting merely because it is self-reinforcing. Useful loops must pay rent. For the abstract chemistry, this means that ACS candidates should be evaluated by interventions: remove them, boost them, shuffle their catalyst links, and compare the resulting dynamics. This is the abstract precursor of causal semantic ACSs.

4 Core scientific questions

The experiments should be organized around concrete questions rather than merely around code milestones.

4.1 Autocatalytic emergence

Can a random or weakly structured catalytic term-rewrite chemistry produce self-sustaining sets of molecules and reactions that satisfy closure and persistence criteria? Are there parameter regimes in which such sets emerge more often than in degree-preserving shuffled controls?

4.2 Evolutionary and ecological dynamics

Once ACSs exist, do they show ecological behavior? We would like to observe birth, death, succession, invasion, parasitism, mutualism, niche specialization, collapse, and recovery. These should not be described only anecdotally; they should be detected from event logs and summarized across seeds.

4.3 Endogenous rule evolution

Can rules themselves become molecules? Can rule-mutating reactions produce variants that survive selection? Do trace-guided mutations outperform random mutations? Do any autocatalytic sets become dependent on evolved rules rather than only on hand-coded bootstrap rules?

4.4 Prediction and active weighting

Can an ActPC-like predictive wrapper improve stability, productivity, or task performance without freezing the soup into a trivial attractor? Can the system learn which reaction families reduce next-window prediction error or improve controlled external reward?

4.5 Causal relevance of ACSs

Do detected ACSs actually matter? A candidate ACS should not be accepted as a scientific result until it passes paired replay or ablation tests. Useful metrics include reward drop under ablation, productivity drop, persistence change, diversity change, and resistance to parasites.

5 Recommended starting chemistry

The first chemistry should be a typed catalytic multiset rewrite chemistry. The state of a chamber at tick t is a bounded multiset of molecule ids with abundances:

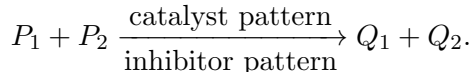
$$S_t(C) = \{(m_i, a_i)\}_{i=1}^n.$$

Each molecule has one or more typed views. Initially the most important view is a token sequence:

$$m = [x_1, x_2, \dots, x_k], \quad x_i \in \Sigma, \quad k \leq L_{max}.$$

Later views may include trees, rule bodies, compressed macro-molecules, semantic graphs, or musical motifs.

A reaction rule has the schematic form:



Here P_i and Q_i are patterns over molecule views, not necessarily concrete molecule ids. The first version may restrict all reactions to one or two reactants and one or two products. This restriction is useful because it makes matching, event logging, and hypergraph reconstruction simple.

5.1 Initial reaction families

The following reaction families are sufficient for the first major experiments:

Family	Role
concat	Combines two short token sequences into a longer sequence, subject to capacity and length limits.
split	Splits a longer sequence into two shorter sequences. This gives catabolism and prevents irreversible growth.
mutate-token	Changes one token in a sequence. This gives local variation.

copy	Replicates a molecule under catalytic conditions. This supports run-away amplification and parasite tests.
compress	Creates a macro-molecule from a repeated or high-frequency motif.
decompress	Re-expands a macro-molecule, allowing macro-level ACSs to interact with ordinary molecules.
rule-mutate	Produces a trial rule molecule by modifying an existing rule. This is introduced only after Experiments 0–2.
rule-compose	Creates a rule from two rules when the products of one match the reactants of the other. This is introduced after rule molecules exist.

5.2 Catalysis and inhibition

Catalysis should start simple but not arbitrary. A molecule can catalyze a rule when its token sequence or macro label matches a catalyst pattern associated with that rule. Examples:

A sequence containing motif M catalyzes rules that produce M.
 A repeated subpattern catalyzes copy and compress rules.
 A palindrome-like sequence catalyzes reverse or split rules.
 A macro molecule catalyzes reactions using its component motif.
 A parasite marker inhibits repair or closure reactions.

The scoring function for a candidate reaction application can initially be:

$$s(R, b, C, t) = \ell_R + \beta_c Cat(R, b, C, t) - \beta_i Inhib(R, b, C, t) - \lambda Cost(R, b) + \beta_n Novelty(R, b, C, t),$$

where R is the rule, b is the binding, C is the chamber, ℓ_R is the rule logit, Cat is the catalytic support, $Inhib$ is inhibition, $Cost$ is length or compute cost, and $Novelty$ is a bounded novelty bonus. The novelty term should be small. A large novelty term will create novelty addiction.

6 PeTTa-first architecture

The first implementation should have four layers.

6.1 Layer A: Atomspace state

The Atomspace stores molecule definitions, abundances, rules, chambers, transient candidates, selected reactions, event logs, inferred catalysis links, ACS annotations, snapshots, and experiment metadata. It should also store enough information to recreate the run from a seed and a config file.

6.2 Layer B: Phase engine

A chamber tick should be explicit and inspectable:

```
tick:
  begin_tick
  inflow_food
  decay_and_dilution
  derive_active_signatures
  match_reaction_candidates
```

```
score_candidates
sample_reactions
fire_reactions
log_events
update_abundances
mine_catalysis_window
scan_acs_window
structural_mutation_step
cleanup_transients
end_tick
```

Not every phase must be implemented as pure PeTTa rewriting. However, every phase should read and write atoms through a narrow adapter so that the same protocol can later be handled by MORCK or host-side kernels.

6.3 Layer C: Analytics

The analytics layer computes catalytic strengths, ACS candidates, persistence, diversity, ecological events, and intervention impact. It should write results back into the Atomspace as atoms. This is crucial: the chemistry should be able to react to its own analytics. For example, a persistent ACS may become a macro-molecule, a parasite motif may become an inhibitor, and a high-impact ACS may bias rule mutation.

6.4 Layer D: Replaceable kernel boundary

Even in the PeTTa-only version, expensive or stochastic operations should be represented as request atoms. The first implementation may answer them in simple local code, but the calling convention should not assume that.

```
(sample-request C T)
(score-request C T)
(acs-scan-request C T W)
(catalysis-scan-request C T W)
(replay-ablation-request C T Snapshot ACS Seed)
(canon-hash-request Obj)
(rule-distance-request R1 R2)
```

Later, these requests can be answered by MORCK, host kernels, parallel workers, or specialized analytics tools without changing the scientific representation.

7 Core atom conventions

Hot-path atoms should be relation-first, with grounded context or object keys early and variables late. The first implementation can be more forgiving than MORCK, but it should train the codebase into MORCK-compatible habits.

7.1 Chambers and ticks

```
(chamber C1)
(chamber-status C1 active)
(chamber-kind C1 abstract-polymer)
(chamber-capacity C1 1000)
(chamber-food-profile C1 food-basic)
(chamber-decay-rate C1 0.01)
(tick C1 T000042)
(seed C1 12345)
```

7.2 Molecules and abundances

```
(mol M17)
(mol-type M17 token-seq)
(seq M17 (A B C))
(seq-len M17 3)
(mol-hash M17 Habc123)
(abund C1 T000042 M17 7)
```

For performance, abundances may also be stored in an external table. But the canonical run state should be representable as atoms or as atom-backed snapshots.

7.3 Rules

```
(rule R10)
(rule-kind R10 reaction)
(rule-status R10 active)
(rule-family R10 concat)
(lhs R10 P_left_1)
(lhs R10 P_left_2)
(rhs R10 P_right_1)
(cat R10 P_cat_1)
(inhib R10 P_inhib_1)
(rule-logit R10 0.0)
(rule-prior-logit R10 0.0)
(rule-cost R10 1.0)
(rule-owner R10 hand-coded)
```

Rules should eventually have both source and compiled forms:

```
(rule-src R10 "human-readable rule source")
(rule-src-hash R10 Hrule10)
(rule-compiled-trigger R10 concat-trigger)
(rule-compiled-guard R10 length-under-limit)
(rule-compiled-template R10 concat-template)
```

The first prototype may treat source and compiled forms as identical, but the fields should be present before the codebase grows.

7.4 Candidates and events

```
(cand C1 T000042 Cand991)
(cand-rule Cand991 R10)
(cand-reactant Cand991 M3)
(cand-reactant Cand991 M4)
(cand-catalyst Cand991 M9)
(cand-product-template Cand991 P_right_1)
(cand-score Cand991 1.37)
```

```
(chosen C1 T000042 Cand991)
```

```
(event E2001)
(event-chamber E2001 C1)
(event-tick E2001 T000042)
(event-rule E2001 R10)
(event-reactant E2001 M3)
(event-reactant E2001 M4)
(event-product E2001 M17)
(event-catalyst E2001 M9)
(event-score E2001 1.37)
(event-seed E2001 12345)
```

Every event must be sufficient for replay analysis. If an event cannot explain which rule fired, with which reactants, catalysts, products, and seed state, it is not an acceptable event.

8 Analytics for autocatalytic sets

The analytics layer turns event logs into scientific claims. There are four levels of analysis.

8.1 Descriptive chemistry metrics

Basic time-series metrics should be emitted for every run:

```
unique molecule count
unique rule count
abundance entropy
reaction entropy
mean molecule length or tree depth
macro molecule count
turnover rate
dominant lineage age
candidate count per tick
runtime per tick
```

These metrics tell us whether the soup is alive, frozen, exploding, or degenerate, but they do not yet prove autocatalysis.

8.2 Empirical catalysis mining

For each molecule m and reaction rule r , estimate an empirical catalytic strength over a recent window W :

$$\kappa(m, r) = \log \frac{P(\text{fire } r \mid m \text{ present}) + \epsilon}{P(\text{fire } r \mid m \text{ absent}) + \epsilon}.$$

A simple confidence value is:

$$\text{conf}(m, r) = \frac{N_{m,r}}{N_{m,r} + k}.$$

The analytics layer writes:

```
(catalyzes M17 R10 1.28 0.64)
(inhibits M23 R10 0.93 0.58)
```

These atoms can later influence candidate scoring, rule mutation, and macro ACS creation. The first version should distinguish explicit catalyst rules from empirically mined catalyst effects. Both are useful, but they answer different questions.

8.3 ACS and RAF-like closure

For each window W , construct a bipartite directed hypergraph:

```
Molecule -> Reaction if molecule is a reactant.
Molecule -> Reaction if molecule is a catalyst.
Reaction -> Molecule if molecule is a product.
Food -> Molecule if molecule is supplied externally.
```

A candidate ACS should satisfy at least the following conditions:

- Reaction closure: reactions in the set produce molecules that support other reactions in the set.
- Catalytic closure: each reaction has a catalyst from the food set or from molecules produced by the set.
- Food reachability: reactants can be built from food molecules plus products of the set.
- Persistence: the set recurs across multiple windows or survives for more than a minimum number of ticks.
- Nontriviality: the set contains more than a single reflexive copy rule or a one-rule artifact unless explicitly marked as a degenerate ACS.

The output should be represented as atoms:

```
(acs ACS42)
(acs-window ACS42 C1 T000100 T000300)
(acs-member-mol ACS42 M17)
(acs-member-rule ACS42 R10)
(acs-food-supported ACS42 true)
(acs-catalytically-closed ACS42 true)
(acs-size-mol ACS42 11)
(acs-size-rule ACS42 6)
(acs-persistence ACS42 200)
(acs-productivity ACS42 3.8)
```

8.4 Causal relevance by replay and ablation

A candidate ACS becomes scientifically interesting when it matters under intervention. Let J be a run-level objective such as productivity, diversity, complexity, target production, prediction improvement, or explicit task reward. For an ACS A , define:

$$\text{Impact}(A) = E[J \mid \text{normal}] - E[J \mid \text{ablate}(A)].$$

The replay protocol should use the same snapshot and seed when possible:

Condition 1: normal replay.
Condition 2: remove ACS member rules.
Condition 3: remove ACS member molecules.
Condition 4: boost ACS member molecules and rules.
Condition 5: degree-preserving catalyst shuffle control.

Output atoms:

```
(acs-causal-test ACS42 ablation reward-drop 0.31 0.44)
(acs-causal-test ACS42 activation reward-gain 0.27 0.39)
(acs-causal-test ACS42 shuffled-control-drop 0.04 0.31)
(acs-clarity ACS42 0.76)
(acs-redundancy ACS42 0.22)
```

The distinction between active ACS and causal ACS should be explicit:

```
active ACS: self-sustaining and recurrent
useful ACS: self-sustaining and associated with reward or productivity
causal ACS: useful ACS whose ablation or activation changes outcomes
```

9 Experimental trajectory

The project should progress through a sequence of experiments that each answer a specific question and leave behind reusable code.

9.1 Experiment 0: deterministic smoke test

This validates schemas, abundance updates, event logging, and replay. It uses a hand-coded toy system with a few molecules and reactions. No emergence is claimed.

9.2 Experiment 1: hand-planted ACS recovery

This validates the ACS detector. A known autocatalytic set is embedded in a soup with distractor reactions. The detector should recover it and ablation should matter.

9.3 Experiment 2: random catalytic polymer chemistry

This is the first emergence experiment. Molecules are token sequences. Reactions include concat, split, mutate, copy, compress, and decay. Catalyst links are partly pattern-based and partly random. The experiment searches for parameter regimes where ACSs emerge more often than in shuffled controls.

9.4 Experiment 3: rule molecules and endogenous rule evolution

Rules become molecules. Rule-mutating reactions generate trial rules. Trial rules are promoted or retired based on recent credit, ACS participation, and cost. This tests whether algorithmic chemistry can alter its own reaction space.

9.5 Experiment 4: ecological chambers, migration, and parasites

Multiple chambers have different food profiles, decay rates, and capacities. Molecules and rules can migrate. Parasites exploit ACS products without contributing to closure. This tests niche formation, invasion, collapse, and recovery.

9.6 Experiment 5: active-predictive weighting

The system predicts the next-window molecule distribution. Rules are rewarded for reducing prediction error and improving controlled objectives. This adds an ActPC-like layer while keeping the substrate abstract.

9.7 Experiment 6: trace-guided structural mutation

Instead of purely random rule mutations, the system uses traces, high-credit ACS members, parasite failures, and high-surprise transitions to propose structural mutations. This is the PeTTa-only precursor to later MOSES/MORK structural proposal machinery.

9.8 Experiment 7: causal ACS validation archive

Detected ACSs are subjected to paired replay, ablation, activation, and shuffled controls across many seeds. The output is an archive of causal ACSs, degenerate loops, parasites, and failed candidates.

10 Expected outcomes and implications

A successful first phase would not prove AGI, musical creativity, semantic reasoning, or biological chemical realism. It would establish something more specific and more useful: a working experimental substrate in which symbolic rewrite molecules can form measurable, replayable, causally testable reaction networks.

That substrate has several implications.

- For cognitive architecture, it provides a concrete testbed for self-modifying rule ecologies before they are connected to language or perception.
- For MORK integration, it identifies which operations deserve acceleration: candidate matching, stochastic sampling, window analytics, replay, and structural rule proposal.
- For semantic chemistry, it clarifies which ACS analytics are domain independent and which require graph-specific semantics.
- For music, it gives a path from abstract motif chemistry to phrase, rhythm, harmony, and style chemistries.

- For coding-agent development, it gives a sequence of testable milestones rather than a vague research program.

A negative result is also informative. If the abstract system fails to produce nontrivial ACSs except under heavy hand coding, then the design should be simplified further, the reaction family should be revised, or the analytics may be detecting the wrong structures. Failure should be logged with the same seriousness as success.

11 Later path to semantic chemistry

Semantic graph chemistry should be introduced only after the abstract substrate has working event logs, ACS mining, ablation, and multi-chamber dynamics. The mapping is straightforward:

Abstract chemistry	Semantic chemistry
Token sequence molecule	Semantic graph fragment or FUSE-NF-like molecule
Catalytic rewrite rule	Contextual semantic graph transformation
Chamber	Discourse, domain, question, or world context
Food	Parsed sentences, KB facts, primitive roles, question graph
ACS	Self-sustaining semantic inference or generation loop
Reward	QA score, explanation compactness, hypothesis utility, narrative power
Ablation	Rule, module, chamber, or ACS intervention

The main lesson to carry over is not a specific token rule. It is the discipline of event logs, closure detection, intervention testing, and treating inference control as something that evolves inside the chemistry.

12 Later path to music chemistry

Music chemistry should also wait until abstract ACS dynamics work. The first music version should use symbolic motifs rather than audio. A possible mapping is:

Abstract chemistry	Music chemistry
Token sequence	Pitch, rhythm, interval, chord, or phrase motif
Concat	Phrase concatenation or rhythmic extension
Split	Motif extraction or phrase segmentation
Mutate-token	Pitch, duration, articulation, or harmony variation
Copy	Repetition, sequence, ostinato, motivic recall
Compress	Theme abstraction or style macro
Catalyst	Key, meter, groove, harmonic context, instrumentation, style marker
ACS	Self-sustaining compositional idiom or motif ecology
Reward	Coherence, novelty, tension-release, style fit, human or model score

The first music experiment should not try to generate finished songs. It should ask whether motif-rewrite chemistries form persistent, rewarded ACSs that produce coherent phrase families.

Finished composition can come later.

13 Recommended implementation governance

This project is well suited to an agentic development process, but only if the agent is constrained by tests and artifacts. A coding agent should not be asked to “implement algorithmic chemistry” in one step. It should be given the appendix as a milestone ladder.

A practical workflow is:

- An OpenClaw-style coding agent performs routine implementation, refactoring, tests, and experiment automation.
- GPT-5.5-level coding assistance is used for ordinary software tasks: schema files, adapters, CLI runners, tests, plotting, and parameter sweeps.
- GPT-5.5-Pro-level strategic review is called at milestone boundaries: after Experiment 0, after planted ACS recovery, after first random chemistry sweeps, after rule evolution, and before semantic or music extensions.
- Every strategic review receives the same package: config, seed list, event-log summary, metrics CSV, ACS archive, ablation results, known failures, and next proposed changes.

The review question should not be “does the code look clever?” It should be: what did the experiment demonstrate, what did it fail to demonstrate, and which next change most increases scientific information per engineering hour?

A Appendix: coding-agent implementation plan

This appendix is written for coding agents. It is deliberately operational. The first instruction is: do not implement the whole vision. Implement the smallest slice that produces reproducible event logs and one validated ACS detector.

A.1 Non-goals for version 0.1

Version 0.1 should not implement semantic graphs, MORK, MOSES, PLN, AIRIS, optimal transport, music generation, neural models, or an open-ended GUI. It should not optimize prematurely. It should not invent a giant nested molecule format that prevents later indexing.

Version 0.1 should implement:

1. Atom representation and query helpers.
2. Chamber configuration and seeded random generator.
3. Molecule and rule schemas.
4. Deterministic tick loop.
5. Stochastic reaction sampling.
6. Event logging and snapshot export.
7. Planted ACS detector.
8. Basic metrics and CSV/JSON output.
9. Minimal command-line runner.
10. Tests for deterministic replay.

B Repository layout

Use a layout that separates experiment logic, PeTTa artifacts, analytics, and host utilities.

```
petta-chem/  
  README.md  
  docs/  
    design.md  
    experiment_log_template.md  
  petta/  
    schemas.metta  
    phases.metta  
    bootstrap_rules.metta  
    analytics_rules.metta  
  src/  
    atomspace/  
      mod.rs or atomspace.py  
      atom.rs  
      query.rs  
      serialize.rs  
    chemistry/  
      molecule.rs  
      rule.rs  
      chamber.rs
```

```

    tick.rs
    reactions.rs
    sampling.rs
analytics/
    catalysis.rs
    acs.rs
    raf.rs
    ablation.rs
    metrics.rs
experiments/
    exp00_smoke.rs
    exp01_planted_acs.rs
    exp02_random_polymer.rs
    exp03_rule_molecules.rs
    exp04_ecology.rs
    exp05_predictive.rs
    exp06_structural.rs
    exp07_causal_archive.rs
cli/
    run.rs
    sweep.rs
    replay.rs
    report.rs
configs/
    exp00_smoke.toml
    exp01_planted_acs.toml
    exp02_random_polymer.toml
tests/
    test_atoms.*
    test_replay.*
    test_acs_detector.*
    test_sampling.*
outputs/
    .gitkeep

```

Rust, Python, or another implementation language may be used for host kernels, but the atom protocol and PeTTa files should remain central. Every host function should have a pure unit-testable core and a thin atomspace adapter.

C Atom schema checklist

Implement these atom families before adding complex experiments.

C.1 Chamber atoms

```

(chamber C)
(chamber-status C active)
(chamber-kind C abstract-polymer)

```

```
(chamber-capacity C 1000)
(chamber-param C decay-rate 0.01)
(chamber-param C inflow-rate 2.0)
(chamber-param C max-molecule-len 8)
(chamber-param C temperature 1.0)
(tick C T)
(seed C Seed)
```

C.2 Molecule atoms

```
(mol M)
(mol-type M token-seq)
(seq M (A B C))
(seq-len M 3)
(mol-hash M H)
(mol-parent M Parent reaction E)
(abund C T M Count)
```

C.3 Rule atoms

```
(rule R)
(rule-kind R reaction)
(rule-status R active)
(rule-family R concat)
(rule-owner R hand-coded)
(rule-cost R 1.0)
(rule-logit R 0.0)
(rule-prior-logit R 0.0)
(lhs R Pattern1)
(lhs R Pattern2)
(rhs R Pattern3)
(cat R PatternCat)
(inhib R PatternInhib)
(rule-src-hash R H)
```

C.4 Candidate and event atoms

```
(cand C T Cand)
(cand-rule Cand R)
(cand-reactant Cand M)
(cand-catalyst Cand M)
(cand-score Cand Score)
(chosen C T Cand)
```

```
(event E)
(event-chamber E C)
(event-tick E T)
(event-rule E R)
(event-reactant E M)
```

```

(event-product E M)
(event-catalyst E M)
(event-score E Score)
(event-seed E Seed)

```

C.5 Analytics atoms

```

(catalyzes M R Strength Confidence)
(inhibits M R Strength Confidence)
(acs A)
(acs-window A C T0 T1)
(acs-member-mol A M)
(acs-member-rule A R)
(acs-food-supported A true)
(acs-catalytically-closed A true)
(acs-persistence A N)
(acs-productivity A X)
(acs-causal-test A ablation Metric Delta Confidence)
(acs-clarity A X)
(acs-redundancy A X)

```

D Top-level tick pseudocode

The tick loop should be deterministic given a seed and a state snapshot.

```

procedure RUN_TICK(space, chamber, tick, rng):
    add(space, (tick chamber tick))

    INFLOW_FOOD(space, chamber, tick, rng)
    APPLY_DECAY_AND_DILUTION(space, chamber, tick, rng)
    DERIVE_ACTIVE_SIGNATURES(space, chamber, tick)

    candidates = MATCH_REACTION_CANDIDATES(space, chamber, tick)
    scored = SCORE_CANDIDATES(space, chamber, tick, candidates)
    chosen = SAMPLE_REACTIONS(space, chamber, tick, scored, rng)

    events = FIRE_REACTIONS(space, chamber, tick, chosen)
    UPDATE_ABUNDANCES(space, chamber, tick, events)
    LOG_EVENTS(space, chamber, tick, events)

    if tick % catalysis_scan_period == 0:
        MINE_CATALYSIS(space, chamber, tick, window)

    if tick % acs_scan_period == 0:
        SCAN_ACS(space, chamber, tick, window)

    if tick % structural_period == 0:
        STRUCTURAL_MUTATION_STEP(space, chamber, tick, rng)

```

```

    CLEANUP_TRANSIENTS(space, chamber, tick)
end procedure

```

E Candidate matching guidance

For version 0.1, brute-force matching is acceptable if molecule counts are small. However, the interface should already look like a future indexed matcher.

```

procedure MATCH_REACTION_CANDIDATES(space, C, T):
    active_rules = query(rule-status R active)
    active_mols = query(abund C T M Count where Count > 0)
    candidates = []

    for R in active_rules:
        reactant_bindings = match_lhs_patterns(R, active_mols)
        for b in reactant_bindings:
            catalysts = match_catalyst_patterns(R, active_mols, b)
            inhibitors = match_inhibitor_patterns(R, active_mols, b)
            if catalysts not empty and inhibitors acceptable:
                Cand = allocate_candidate_id()
                write_candidate_atoms(Cand, R, b, catalysts, inhibitors)
                candidates.append(Cand)

    cap candidates by chamber and rule-family budgets
    return candidates
end procedure

```

Do not let candidate count grow without a budget. Add hard caps early:

```

max_candidates_per_tick = 512
max_candidates_per_rule = 32
max_candidates_per_family = 128

```

F Sampling guidance

Use a stable softmax with temperature and seeded random numbers.

```

procedure SAMPLE_REACTIONS(scored_candidates, rng):
    if scored_candidates is empty:
        return []

    group candidates by chamber or reaction slot
    chosen = []
    for each group:
        max_score = max(score(c) for c in group)
        weight(c) = exp((score(c) - max_score) / max(temp, eps))
        selected = categorical_sample(group, weights, rng)

```

```

        chosen.append(selected)
    return chosen
end procedure

```

Every sampling test should check seed reproducibility. If the same config and seed produce different event logs, stop and fix replay before adding features.

G ACS scanner pseudocode

The first scanner can be conservative. It is better to miss some ACSs than to label every cycle as an ACS.

```

procedure SCAN_ACS(space, C, T, window):
    events = read_events(C, T-window, T)
    graph = build_bipartite_graph(events)

    components = strongly_connected_components(graph)
    candidates = []

    for comp in components:
        if count_rules(comp) < min_rules:
            continue
        if count_molecules(comp) < min_molecules:
            continue

        closure = RAF_CLOSURE(comp, food_set(C), graph)
        if closure.catalytically_closed and closure.food_supported:
            A = allocate_acs_id()
            write_acs_atoms(A, C, T-window, T, closure)
            candidates.append(A)

    score_persistence_and_productivity(candidates)
    return candidates
end procedure

procedure RAF_CLOSURE(component, food, graph):
    available = set(food)
    chosen_rules = empty set
    changed = true

    while changed:
        changed = false
        for rule in component.rules:
            if rule in chosen_rules:
                continue
            if reactants(rule) subset available and catalyst_supported(rule, available):
                chosen_rules.add(rule)
                available.add_all(products(rule))

```

```

        changed = true

# Prune unsupported rules until fixed point.
changed = true
while changed:
    changed = false
    for rule in copy(chosen_rules):
        if not reactants(rule) subset available:
            chosen_rules.remove(rule)
            changed = true
        if not catalyst_supported(rule, available):
            chosen_rules.remove(rule)
            changed = true
    available = food union products(chosen_rules)

    return closure(chosen_rules, available)
end procedure

```

H Ablation pseudocode

Ablation is expensive, so it should run on selected ACS candidates rather than every window.

```

procedure TEST_ACS_CAUSALITY(snapshot, ACS, seed, config):
    normal = REPLAY(snapshot, seed, config)

    cfg1 = config with ACS member rules disabled
    ablate_rules = REPLAY(snapshot, seed, cfg1)

    cfg2 = config with ACS member molecules removed or clamped low
    ablate_mols = REPLAY(snapshot, seed, cfg2)

    cfg3 = config with ACS member rules and molecules boosted
    boost = REPLAY(snapshot, seed, cfg3)

    cfg4 = config with catalyst links shuffled preserving degrees
    shuffle = REPLAY(snapshot, seed, cfg4)

    impact_rules = metric(normal) - metric(ablate_rules)
    impact_mols = metric(normal) - metric(ablate_mols)
    gain_boost = metric(boost) - metric(normal)
    shuffle_gap = impact_rules - (metric(normal) - metric(shuffle))

    write acs-causal-test atoms
end procedure

```

I Experiment 0: deterministic smoke test

I.1 Goal

Validate atom schemas, deterministic abundance updates, event logging, and snapshot replay.

I.2 Initial molecules

A, B, C, D, E, X

I.3 Hand-coded reactions

R1: $A + B \rightarrow C$

R2: $C + D \rightarrow E$

R3: $E + A \rightarrow C + E$

Catalysis can be disabled for the very first test, then enabled after replay is stable.

I.4 Acceptance criteria

Same seed and config produce byte-identical event logs.

All abundance counts remain nonnegative.

Every chosen candidate has a corresponding event.

Every event has rule, reactants, products, chamber, tick, and seed.

Runtime for 100 ticks is stable and small.

J Experiment 1: planted ACS recovery

J.1 Goal

Validate the ACS detector and ablation protocol before searching for emergence.

J.2 Food set

$F = \{A, B, X\}$

J.3 Planted ACS

R1: $A + B \rightarrow C$ catalyzed by E

R2: $C + X \rightarrow D$ catalyzed by C

R3: $D + B \rightarrow E$ catalyzed by D

R4: $E + A \rightarrow C + E$ catalyzed by C

Add distractor reactions that share some molecules but do not complete the catalytic closure.

J.4 Run matrix

noise_reactions: 0, 5, 20, 100

decay_rate: 0.0, 0.01, 0.05

food_inflow: low, medium, high

capacity: 100, 1000

seeds: 20 per condition

J.5 Metrics

ACS precision and recall against planted set
time to first detection
false positive ACS count
ablation productivity drop
persistence under decay

K Experiment 2: random catalytic polymer chemistry

K.1 Goal

Find parameter regimes where non-planted ACSs emerge more often than in shuffled controls.

K.2 Molecule space

alphabet_size: 4, 6, 8
max_length: 4, 6, 8, 12
initial_food: all length-1 tokens plus selected length-2 tokens

K.3 Reaction families

concat: $X + Y \rightarrow XY$
split: $XY \rightarrow X + Y$
mutate: $X \text{ a } Y \rightarrow X \text{ b } Y$
copy: $X \rightarrow X + X$, capacity-limited
compress: repeated motif $\rightarrow$ macro molecule
decay: $X \rightarrow$ fragments or removal

K.4 Catalyst rules

Molecule containing motif M catalyzes production of M.
Molecule with repeated motif catalyzes copy and compress.
Macro molecule catalyzes reactions involving its motif.
Random catalyst links are added with probability p_{cat} .

K.5 Controls

Control A: catalyst links shuffled preserving molecule degrees.
Control B: product identities shuffled preserving rule family counts.
Control C: catalysis disabled, same food and decay.
Control D: event timestamps shuffled before ACS scan.

K.6 Success criteria

A run condition is interesting if ACSs are more frequent, larger, more persistent, or more intervention-relevant than in controls, and if the effect replicates across seeds.

L Experiment 3: rule molecules

L.1 Goal

Allow the chemistry to alter its own reaction set.

L.2 Rule molecule atoms

```
(mol R10)
(mol-type R10 rule)
(rule R10)
(rule-status R10 trial)
(rule-parent R10 R3 mutate-catalyst)
(rule-src-hash R10 Hnew)
```

L.3 Rule mutation operators

```
mutate_lhs_token
mutate_rhs_token
add_catalyst_pattern
remove_catalyst_pattern
change_rule_family
specialize_by_context
compose_rules
split_rule
```

L.4 Trial promotion

```
procedure PROMOTE_OR_RETIRE_TRIAL_RULES(space):
  for R in trial rules:
    if use_count(R) < min_trial_uses:
      continue
    if duplicate_of_active_rule(R):
      retire R
    else if avg_credit(R) > promote_threshold and cost_ok(R):
      promote R to active
    else if avg_credit(R) < retire_threshold:
      retire R
end procedure
```

L.5 Metrics

```
trial rule survival rate
promoted rule fraction
rule lineage depth
ACSs containing evolved rules
rule bloat rate
duplicate rule rate
trace-guided mutation yield versus random mutation yield
```

M Experiment 4: ecological chambers and parasites

M.1 Goal

Observe multi-chamber ecological dynamics.

M.2 Chamber setup

C1: food profile rich in A/B motifs, low decay
C2: food profile rich in C/D motifs, medium decay
C3: sparse food, high decay, high migration benefit
migration rates: 0.0, 0.001, 0.01, 0.05

M.3 Parasite definition

A parasite is a molecule or rule that benefits from an ACS without supporting its closure. Operationally:

P consumes or copies products of ACS A.
P is catalyzed by molecules in ACS A.
P does not produce reactants or catalysts needed by A.
Ablating P improves or preserves A’s persistence or productivity.

M.4 Metrics

ACS birth and death events
migration success rate
niche specialization index
parasite load
collapse events
recovery time after invasion
cross-chamber transfer of ACS motifs

N Experiment 5: active-predictive wrapper

N.1 Goal

Add prediction-error-based weighting without introducing semantic graphs.

N.2 Prediction target

Predict the next-window molecule-family distribution:

$$p_t(m) = P(m \text{ appears in } [t+1, t+W] \mid S_t, R_t).$$

Observed distribution:

$$q_t(m) = \text{normalized actual abundance or event count in } [t+1, t+W].$$

Prediction error:

$$e_t = D_{KL}(q_t \parallel p_t).$$

N.3 Reward

$$r_t = \alpha_{env} r_t^{env} - \alpha_{err} e_t + \alpha_{lp} \max(0, e_{t-1} - e_t) + \alpha_{acs} ACSYield_t.$$

N.4 Simple logit update

$$\ell_R \leftarrow (1 - \rho)\ell_R + \rho\ell_R^{prior} + \eta \text{clip}(A_R, -a_{max}, a_{max}) - \lambda \text{cost}(R).$$

N.5 Acceptance criteria

Predictive weighting should improve prediction or controlled task reward without collapsing molecule diversity below a configured threshold. If diversity collapses, reduce α_{err} , increase entropy regularization, or add capacity-preserving exploration.

O Experiment 6: trace-guided structural mutation

O.1 Goal

Use analytics to propose better rule mutations.

O.2 Evidence sources

high-credit rules
persistent ACSs
ACS collapse traces
parasite invasion traces
high-surprise windows
rules with high duplicate pressure
rules with high bloat cost

O.3 Mutation proposals

Create rule that produces missing catalyst.
Add inhibitor against parasite motif.
Compress repeated ACS motif into macro molecule.
Split overgeneral rule by chamber context.
Merge duplicate or near-duplicate rules.
Add catalyst pattern from empirical catalysis mining.
Remove catalyst pattern that fails ablation.

O.4 Comparison

Every trace-guided mutation batch should be compared with a same-size random mutation batch under the same trial budget.

P Experiment 7: causal ACS archive

P.1 Goal

Create a reusable archive of ACS candidates and intervention results.

P.2 Archive record

acs_id
config_hash
seed
snapshot_tick
member_molecules
member_rules
closure_metrics
persistence_metrics
normal_replay_metrics
ablation_metrics
activation_metrics
shuffle_control_metrics
classification: causal, useful-noncausal, degenerate, parasite, false-positive
notes

P.3 Scientific report template

For each accepted causal ACS, generate a report:

1. What molecules and rules constitute the set?
2. How was it detected?
3. How long did it persist?
4. What does it produce?
5. Which catalysts close the loop?
6. What happens under ablation?
7. What happens under activation?
8. What controls were run?
9. Does it replicate across seeds?
10. What ecological role does it appear to play?

Q Metrics file formats

Each run should produce at least these files:

run_config.toml
run_manifest.json
events.jsonl
abundances.parquet or abundances.csv
metrics_timeseries.csv
acs_candidates.jsonl
acs_ablation_results.jsonl
rule_lineages.jsonl
summary.md

The summary should include:

config hash
seed list
number of ticks
final molecule diversity
final rule diversity
ACS candidates found
causal ACSs found
largest ACS size
longest ACS persistence
best ablation impact
parasite events
trial rule promotion rate
prediction KL slope, if enabled
known failures

R Testing checklist

R.1 Unit tests

Atom parser round trip.
Molecule canonical hash stable across runs.
Rule canonical hash stable across equivalent source forms.
Softmax sampling deterministic under seed.
Abundance updates conserve counts when configured to conserve.
Decay never creates negative abundance.
Candidate caps are enforced.
ACS scanner recovers planted ACS.
ACS scanner rejects simple non-catalytic cycle when catalyst support is absent.
Ablation replay uses identical seed and snapshot.
Trial duplicate rule is retired.

R.2 Integration tests

Run exp00 for 100 ticks and compare event log to golden file.
Run exp01 and recover planted ACS with precision above threshold.
Run exp02 small sweep and produce metrics files.
Replay a snapshot and reproduce future events under same seed.
Run ablation on planted ACS and observe configured productivity drop.

S OpenClaw and model-review workflow

A coding agent should work in short branches. Each branch should have one experiment or one infrastructure target. A suggested process:

1. Implement feature behind tests.
2. Run unit tests.
3. Run smallest relevant experiment.
4. Save outputs and summary.

5. Ask strategic reviewer only at checkpoint boundaries.
6. Incorporate review into next branch.

Strategic review packages should include:

README for the milestone
 config files
 seed list
 git diff summary
 test results
 metrics plots or CSV summary
 ACS reports
 ablation results
 failure notes
 specific questions for review

Suggested review checkpoints:

Checkpoint A: after Experiment 0 deterministic replay.
 Checkpoint B: after Experiment 1 planted ACS recovery.
 Checkpoint C: after first Experiment 2 parameter sweep.
 Checkpoint D: after rule molecules survive trial promotion.
 Checkpoint E: after parasite/ecology dynamics appear.
 Checkpoint F: before adding predictive weighting.
 Checkpoint G: before semantic chemistry or music chemistry.

T MORK migration seams

Keep these seams explicit from the beginning.

Subsystem	Later MORK or host acceleration target
Atom storage	Prefix-sensitive indexed storage of relation-first facts
Candidate matching	Compiled trigger/guard/template facts and hot-path matching
Sampling	Grounded stochastic sampler with seed discipline
Scoring	Numeric kernel for candidate scores and costs
ACS scan	Bulk graph analytics over event windows
Replay	Snapshot and deterministic event replay engine
Canonical hash	Rule and molecule hash-consing
Rule distance	Structural and behavioral neighbor graph for rule space
Structural mutation	MOSES or other slower proposal machinery

Do not wait until the port to define these boundaries. The first PeTTa version should already behave as if these calls could be remote or accelerated later.

U Failure modes and mitigations

Failure mode	Symptom	Mitigation
Candidate explosion	Thousands of candidates per tick	Active signatures, per-rule caps, per-family budgets, decay, retired status.
Trivial ACSs	Detector reports one-rule copy loops	Minimum size, catalyst closure, ablation requirement, degenerate label.
Novelty addiction	Soup creates junk variety	Small novelty coefficient, learning-progress reward, cost penalty.
Frozen attractor	One molecule or rule dominates	Temperature floor, decay, capacity pressure, migration, random inflow.
Rule bloat	Rule molecules grow without benefit	Cost penalty, trial retirement, duplicate hashing, max rule size.
False catalysis	Correlation mistaken for catalysis	Confidence threshold, shuffled controls, paired replay.
Parasite takeover	Productive ACS collapses	Detect parasite load, evolve inhibitors, chamber migration, diversity reserves.
Unreplayable runs	Same seed gives different logs	Fix all RNG paths, snapshot all state, make parallelism deterministic or disable it.
Premature MORK coupling	Prototype slows under integration burden	Keep MORK as interface seam only until abstract chemistry works.

V Minimal configuration examples

V.1 Experiment 0

```

experiment = "exp00_smoke"
seed = 12345
ticks = 100
chambers = 1
capacity = 100
food_inflow_rate = 0.0
decay_rate = 0.0
temperature = 0.0
stochastic = false
acs_scan_enabled = false

```

V.2 Experiment 1

```

experiment = "exp01_planted_acs"
seed = 12345
ticks = 500
capacity = 500
food = ["A", "B", "X"]
food_inflow_rate = 2.0
decay_rate = 0.01

```

```
temperature = 0.5
noise_reactions = 20
acs_scan_period = 25
acs_window = 100
ablation_enabled = true
```

V.3 Experiment 2

```
experiment = "exp02_random_polymer"
seeds = 100
alphabet_size = [4, 6, 8]
max_length = [4, 6, 8]
capacity = [100, 1000]
decay_rate = [0.001, 0.01, 0.05]
food_inflow_rate = [0.5, 2.0, 10.0]
temperature = [0.2, 0.7, 1.5]
random_catalyst_probability = [0.01, 0.05, 0.10]
controls = ["shuffle_catalysts", "shuffle_products", "no_catalysis"]
```

W Stop conditions for coding agents

A coding agent should stop and request review when any of the following occurs:

1. Deterministic replay fails.
2. Candidate counts grow superlinearly in small tests.
3. ACS detector reports many ACSs in shuffled controls.
4. Experiment 1 planted ACS is not recovered.
5. Rule mutation creates duplicate floods.
6. Metrics are not interpretable from saved outputs.
7. The code requires semantic graphs or MORK to pass abstract tests.
8. Any feature cannot be unit-tested without a full run.

X Bridge to semantic and music versions

After Experiment 7, create two small bridge prototypes rather than immediately building full applications.

X.1 Semantic bridge prototype

Replace token sequence molecules with tiny hand-authored graph motifs. Keep the same reaction engine and ACS scanner. Do not use open-domain parsing. Test whether the same ACS analytics work on graph-shaped molecules.

X.2 Music bridge prototype

Replace token sequences with symbolic motifs:

(pitch-class sequence)
(interval sequence)
(rhythm sequence)
(chord-function sequence)

Use simple transformations: repeat, invert, transpose, stretch rhythm, cadence, and motif compression. Reward should initially be rule-based: bounded repetition, clear motif recurrence, phrase length, and avoidance of degenerate loops. Human or model aesthetic judgment can be added after structural ACSs are visible.

Y Bibliographic pointers

References

- [1] Ben Goertzel and collaborators. *A Software Design for Algorithmic Chemistry in MeTTa + MM2 Subroutines on MORK: Weighted Rewrite-Rule Chemistry, Discrete Active Predictive Coding, and MOSES-Based Structural Evolution*. Draft design document, June 25, 2026.
- [2] Ben Goertzel and GPT-5.5 Pro. *Semantic Graph Algorithmic Chemistry*. Draft concept document, May 7, 2026.
- [3] Wim Hordijk and Mike Steel. Autocatalytic sets and RAF theory. General background on reflexively autocatalytic and food-generated reaction networks.
- [4] Walter Fontana and Leo Buss. Algorithmic chemistry and artificial chemistries. General background on symbolic reaction systems and emergent organization.